Software Praktikum – Milestone 0

Milestone Health Services App Front End – Functionalities & Plan per milestone

1. Background and Purpose

The goal of the Health Services App is to connect patients with doctors through a digital platform. Patients and doctors will both access the Health Services App through a unified responsive web interface. Patients will use it primarily on mobile browsers, while doctors will access it from desktop devices.
The application is built entirely with Next.js (React + TypeScript), ensuring consistent design, shared logic, and fast iteration.

Doctors will use a web interface to manage their schedules and view patient appointments.

This document covers only the front-end behaviour for both patient and doctor interfaces, the implementation of the backend/API will be implemented by another team member.

2. Technologies

Layer	*Technology*	*Reason*
Frontend (unified web)	TypeScript + React (Next.js)	Single responsive web app for both patient and doctor; modern web tooling and fast iteration
Version control	GitHub	Centralized repo for code and documentation
Design Tools	MS word prototypes	Visual prototypes and user flows

3. Functional requirements

A. Authentication and User Session (shared)

- **01** Users can **Sign Up** (patient only) by providing name, email, and password.
- **02** Users can **Sign In** (patients and doctors) with email and password.

- **03** The app stores a session token and reuses it on browser session restart. (In later milestones, we will refine this to a more secure storage mechanism appropriate for the environment.)
- **04** Users can **Sign Out**; session is cleared from the device/browser.
- **05** Show validation errors for invalid inputs and API errors (401/400).

B. Patient Interface (Responsive Web)

Home & Doctor Discovery

- **01** Patient sees a list of doctors with name, specialty and city.
- **02** Patient can filter/search doctors by specialty and city.
- **03** Patient can open a doctor profile to view the details.

Availability & Booking

- **04** Patient can view a doctor's **available time slots** for a selected date.
- **05** Patient can **book an appointment** by selecting a slot and confirming.
- **06** On successful booking, the app shows a **confirmation** screen/state.
- **07** If the slot is no longer available, show a **conflict message** and refresh availability.

My Appointments

- **08** Patient can view a **list of upcoming appointments** (date, time, doctor).
- **09** Patient can view **appointment details** (doctor info, location, notes).
- **10** Patient can **cancel** an upcoming appointment with confirmation.

Profile

- **11** Patient can **view profile** (name, email).
- **12** Patient can **edit profile** (name; email change may require re-auth).

App States & UX Quality

- **13** Show **loading states** while fetching data.
- **14** Show **empty states** when no data (no doctors, no appointments).
- **15** Show **error states** with retry actions on network/API failures.

C. Doctor Interface(Responsive web)

Dashboard

- **01** Doctor can **sign in** and land on a dashboard.
- **02** Doctor sees a **calendar/list of appointments** (today / selected range).
- **03** Doctor can **filter** appointments by date range.

Appointment details

- **04** Doctor can open an **appointment detail** view (patient name, time, reason/notes if provided).
- **05** Doctor can **confirm/cancel** an appointment (status update with feedback).

Patient chart

- **06** Doctor can view a **basic patient card** (name, contact; medical data).

Session states

- **07** Keep doctor **signed in** across sessions until logout/expiry.
- **08** Show loading/empty/error states consistently.

D. Cross-cutting front end behaviour

- Loading, Error, Empty states are defined and will be applied to all lists/details.
- Form validation at login/sign-up (required + email format).
- 401 handling: redirect to Sign In; token stored securely.
- Consistent header titles and primary CTA at the bottom of the screen.
- Date/time shown in device local time.

Plan - Milestone 1

Objective

Set up the technical foundations for the Health Services App: basic frontend structure for both patient and doctor interfaces within a single Next.js project, plus an initial API/data contract between frontends and backend.

Scope

- Initialize a Next.js project with TypeScript.
- Implement routing and placeholder pages for both roles:
- /patient/login, /patient/home, /patient/booking, /patient/appointments, /patient/profile
- /doctor/login, /doctor/dashboard, /doctor/patients
- Display static mock data on key pages (e.g., doctor list, appointments).
- Set up basic layout, shared components (navigation, header/footer), and responsive design.

Backend / API

- Map out the data needed for each planned screen (patient + doctor side).
- Draft an initial API specification (resources, endpoints, request/response shapes) for:
    - Authentication
    - Doctors and search
    - Availability and booking
    - Appointments list

Implementation notes

- Running Next.js app with navigation and placeholder pages.
- `/docs/api-contract.md` (API draft) and `/docs/data-flow.md`.
- Updated milestone plan for next phases.

Deliverables / Artifacts

- **Running the patient section with the navigation structure.**
- Running doctor web section (Next.js) with a basic dashboard.
- Updated milestone plan for the project.

Demo checklist

- Start the app and navigate through all patient and doctor sections.
- Show static mock data and explain the API/data mapping.

Acceptance criteria

- The app builds and runs locally.
- Navigation works across roles.
- Initial API/data documentation exists and matches screens.

Milestone 2 - Connected Core Flows

Objective
Replace mock API with real backend endpoints for the core booking journey. Improve data integrity and UX polish across both user roles within the unified Next.js application.

Scope

- Switch Doctors list + Availability + Booking to real API.
- My Appointments from real API.
- Error handling for server codes (400/401/404/409/500).
- Doctor dashboard shows appointments list and appointment details from real API.

Deliverables / Artifacts

- ENV setup + API base URL docs.
- `/docs/testing-notes.md` (test accounts, test data).

Demo checklist

- Live booking against real backend API (dev/staging).
- Refresh shows updated lists for both user roles.

Acceptance criteria

- Core booking works **without mocks** end-to-end.

- No hard-coded data; all from API.

Milestone 3 - Enhancements & Reliability

Objective
Round out features and quality for a smooth demo and evaluation.

Scope:

- Cancel / confirm appointment if enabled in API (for patient or doctor role).
- Basic patient card (for doctor view).
- Performance hygiene, accessibility pass.
- Session expiry UX (401 redirect), toast/inline messages, empty/edge cases.

Deliverables

- QA build(s) + bug list + known limitations.
- Short demo video (1–2 min).

Acceptance criteria

- All **MVP FRs** demonstrably implemented or explicitly marked out-of-scope.
- No blocker bugs for the main flows.

Milestone 4 – Final demo

Objective
Present a coherent product with clear value and documentation.

Scope

- Polish UI (spacing, contrast, copy).
- Finalize docs: README (run instructions), API mapping, FR coverage table (FR → screen → endpoint).
- Tag final release.

Deliverables

- Live demo, slide deck, final repo state.

Acceptance criteria

- The app demonstrates the **patient booking journey** and the **doctor view** clearly.
- Documentation is complete and reproducible.